

ASSIGNMENT-3

Assignment 3: Functions & Modules in Python

Aim

To implement Python programs utilizing user-defined functions and built-in modules (math) to perform specific mathematical calculations, demonstrating an understanding of modular programming principles in Python².

Tasks to be Done

1. Task 1: Calculate Factorial Using a Function: Define a function to calculate the factorial of a given number using either a loop or recursion³³³.
 2. Task 2: Using the Math Module for Calculations: Write a program to calculate the square root, natural logarithm, and sine of a user-input number using the standard math module⁴⁴⁴.
-

Requirements

- Software: Python Interpreter (version 3.x recommended).
 - Task 1: A user-defined function named factorial⁵.
 - Task 2: Utilization of the math module⁶.
 - Output: Print results clearly, following the specified output formats⁷⁷⁷.
 - Submission: Create a GitHub repository containing two Python scripts (one for Task 1, one for Task 2), and a README.md file⁸.
-

Objectives

- To learn how to define and call a user-defined function in Python⁹⁹.

- To practice implementing iterative (or recursive) logic within a function.
 - To understand how to import and use functions from a built-in Python module (math)¹⁰.
 - To perform basic input/output operations with user interaction¹¹.
 - To structure a program using modular programming concepts.
-

Theory

Functions

A function is a block of organized, reusable code that is used to perform a single, related action¹². Functions provide better modularity for your application and a high degree of code reusing. In Task 1, we define a function to calculate the factorial ($n! = n \times (n-1) \times \dots \times 1$).

Modules

A module is simply a file containing Python definitions and statements. The filename is the module name with the suffix .py. Python has a rich set of built-in modules, like the math module, which provides access to common mathematical functions and constants¹³. We use the import statement to make the functions within a module available in our code.

Task 1: Calculate Factorial Using a Function

Algorithm (Iterative Method)

1. Define a function factorial(n) that accepts one argument, n¹⁴.
2. Initialize a variable result to 1.
3. Check if $n < 0$. If so, handle as an error (factorial is not defined for negative numbers).
4. Check if $n = 0$. If so, return 1 (since $0! = 1$).

5. Loop from $i=1$ up to and including n (or from n down to 1).
6. In each iteration, multiply result by i : $result = result * i$.
7. Return the final result¹⁵.
8. Call the factorial function with a sample number (e.g., 5) and print the output¹⁶.

Code

File: task1_factorial.py

Python

Function definition for factorial

def factorial(n):

"""Calculates the factorial of a non-negative integer n iteratively."""

Validation for negative numbers

if n < 0:

return "Factorial is not defined for negative numbers."

Base case: Factorial of 0 is 1

if n == 0:

return 1

Calculate factorial using a loop (iterative approach)

result = 1

for i in range(1, n + 1):

```
    result *= i # result = result * i
```

```
    return result # Returns the calculated factorial
```

```
# --- Main Program ---
```

```
try:
```

```
    # Ask user for input
```

```
    num_input = input("Enter a number: ") # Prompts for input
```

```
    num = int(num_input)
```

```
    # Call the function with the sample number
```

```
    fact_result = factorial(num) # Calls the factorial function
```

```
    # Print the output
```

```
    if isinstance(fact_result, int):
```

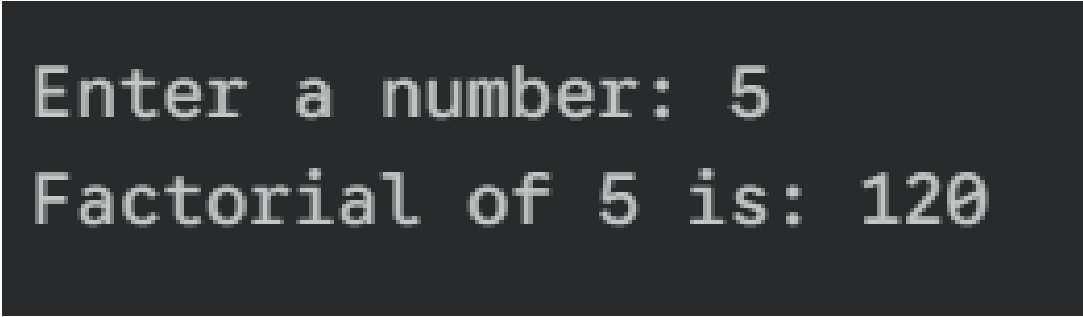
```
        print(f"Factorial of {num} is: {fact_result}")
```

```
    else:
```

```
        print(fact_result)
```

```
except ValueError:
```

```
    print("Invalid input. Please enter a valid integer.")
```



```
Enter a number: 5
Factorial of 5 is: 120
```

Task 2: Using the Math Module for Calculations

Procedure/Steps

1. **Import** the necessary math module¹⁸.
2. **Ask** the user for a number input and **convert** it to a float¹⁹.
3. **Calculate** the square root using `math.sqrt()`²⁰.
4. **Calculate** the natural logarithm using `math.log()` (which is log base e)²¹.
5. **Calculate** the sine using `math.sin()` (The input number is treated as an angle in radians)²².
6. **Display** the calculated results, formatted clearly²³.

Code

File: task2_math_module.py

Python

```
import math # Import the math module
```

```
# --- Main Program ---
```

```
try:
```

```
    # 1. Ask the user for a number as input
```

```
    num_input = input("Enter a number: ") # Asks the user for input
```

```
number = float(num_input)
```

```
# 2. Use the math module to calculate
```

```
# Square root of the number
```

```
square_root = math.sqrt(number)
```

```
# Natural logarithm (log base e) of the number
```

```
# Note: math.log(x) computes the natural logarithm (base e)
```

```
natural_log = math.log(number)
```

```
# Sine of the number (in radians)
```

```
sine_value = math.sin(number)
```

```
# 3. Display the calculated results
```

```
print(f"\nSquare root: {square_root}")
```

```
print(f"Logarithm: {natural_log}")
```

```
print(f"Sine: {sine_value}")
```

```
except ValueError:
```

```
    print("Invalid input. Please enter a valid number.")
```

```
except TypeError:
```

```
    print("Error: Input must be a positive number for log and square root.")
```

except Exception as e:

```
print(f"An error occurred: {e}")
```

```
Enter a number: 25
```

```
Square root: 5.0
```

```
Logarithm: 3.2188758248682006
```

```
Sine: -0.13235175009777303
```